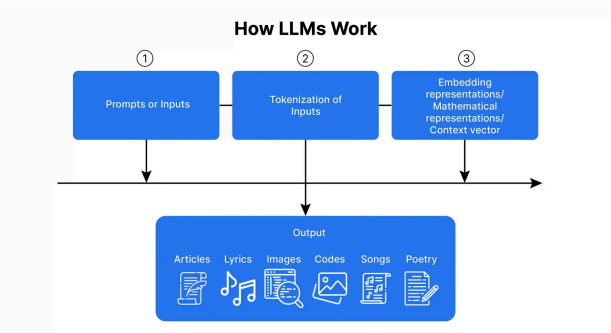
How Transformers Understand Language

Teaching Machines to Understand Language

Have you ever wondered how AI models like ChatGPT understand and generate language? For machines, human language is incredibly complex. To bridge the gap, we use a series of processing steps that convert text into a form computers can work with. These include:



- **Tokenization** – Breaking language into small pieces.
- **Embedding** – Turning those pieces into numbers.
- **Vector Space** – Mapping meaning in a multi-dimensional space.
- **Self-Attention** – Letting the model decide what's important in context.

Together, these steps allow AI to read, understand, and generate natural-sounding language.

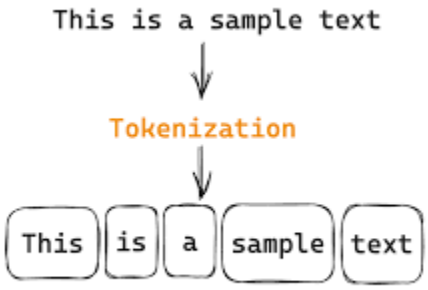
Think of it like reading a recipe. You need to:

- Identify the ingredients (tokenization),
- Understand what each ingredient is (embedding),
- See how they relate in a dish (vector space),
- Focus on the right steps at the right time (self-attention).

Let's break each down.

Tokenization – Breaking Language into Pieces

Machines can't understand sentences the way we do. They first need the text to be split into manageable parts called **tokens**.



What is a Token?

A **token** can be:

- A whole word: *["I", "love", "apples"]*
- A subword or character: *["I", "lov", "e", "app", "les"]*

Different models use different types of tokenization:

- **Word-level** (simple but limited vocabulary)
- **Subword-level** (used in GPT: e.g., Byte-Pair Encoding)
- **Character-level** (very granular, but longer sequences)

> Tokenization is like breaking a LEGO model into pieces. Each piece (token) helps rebuild the full picture later.

Why Tokenization Matters

- Reduces complexity: models don't have to learn from raw text.
- Handles rare or unseen words by splitting them into smaller units.
- Standardizes inputs for models to process.

> Example:
> Sentence: *"I'm learning Transformers!"*
> Tokens (subword-level): `["I", "'m", "learning", "Transform", "ers", "!"]`

Embeddings – Giving Tokens Meaning Through Numbers

Once we have tokens, we need to convert them into a format the model understands — **numbers**. This is where **embedding** comes in.

What is an Embedding?

An **embedding** is a vector — a list of numbers — that represents a token. Each token has a unique embedding that captures its meaning and context.

Example:
Token: `"apple"`
Embedding: `[0.25, -0.13, 0.91, ...]`

These vectors are **learned** during training and reflect how words relate to each other.

> Think of embeddings like flavor profiles. “Apple” and “Pear” may both be fruity and sweet, so they sit close together in flavor space (or vector space).

Benefits of Embeddings

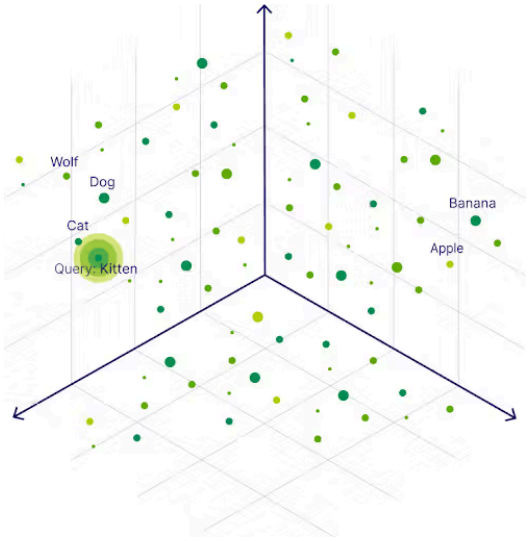
- Capture meaning and context.
- Reflect word similarity.
- Efficient for computation.

> Example:
> Words like `"king"` and `"queen"` will have similar embeddings, but slightly differ by gender.
> In fact, the classic example:
> `"king" - "man" + "woman" ≈ "queen"`
> shows how powerful embeddings can be.

Vector Space – Mapping Relationships Between Words

All embeddings exist in a **vector space**, a kind of high-dimensional map where each word is a point.

What is a Vector Space?



A **vector space** is where we can compare word meanings based on their direction and distance from each other.

- Similar words are **close together**.
- Unrelated words are **far apart**.

Imagine the night sky. Each star is a word, and words that are conceptually related form constellations — groups that make sense together.

Why Vector Space Matters

- Enables models to understand **semantic similarity**.
- Helps the model generalize (e.g., understanding synonyms).
- Forms the basis for reasoning and analogies.

Key Insight: The model doesn't understand text like we do — it understands **patterns in numbers** that represent words.

Self-Attention – Focusing on What Matters

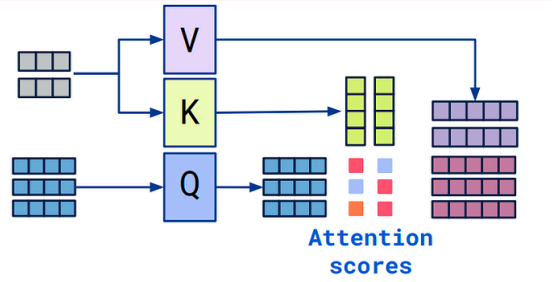
Even with embeddings, not all words in a sentence are equally important. The model needs a way to focus on **relevant** words when understanding a sentence. That's where **self-attention** comes in.

What is Self-Attention?

Self-attention is a mechanism that allows a model to **look at all tokens at once** and decide how much attention to give each one when processing a specific word.

In a conversation, if someone says:
"The trophy doesn't fit in the suitcase because it is too small,"
You need to figure out what "it" refers to. Self-attention helps the model focus on the correct noun ("suitcase") to resolve the reference.

How It Works



Each token gets transformed into:

- **Query (Q)** – what it's looking for
- **Key (K)** – what it has to offer
- **Value (V)** – the information it contains

The model compares the **query** of one word with the **keys** of all other words to compute **attention scores**, which determine how much focus to place on each token's value.

Example: In the sentence: *"The cat sat on the mat."*
When analyzing "sat," the model may attend most to "cat" and "mat" to understand the full action and context.

Why Self-Attention is Powerful

- It captures **long-distance relationships** in text.
- It's **parallelizable** (unlike RNNs).
- It's the core innovation behind **transformers**.

To summarize, transformers understand language through a layered process:

1. **Tokenization** breaks text into pieces.
2. **Embedding** turns those pieces into numbers.
3. **Vector space** gives structure to meaning.
4. **Self-attention** lets the model focus on relevant information in context.

Together, these components allow models to understand, reason, and generate language — powering everything from chatbots to translation tools, coding assistants to image generators.